

Torchair

昇腾NPU亲和的torch.compile后端

Peng Xue @medivh-xp
medivh-xp@outlook.com

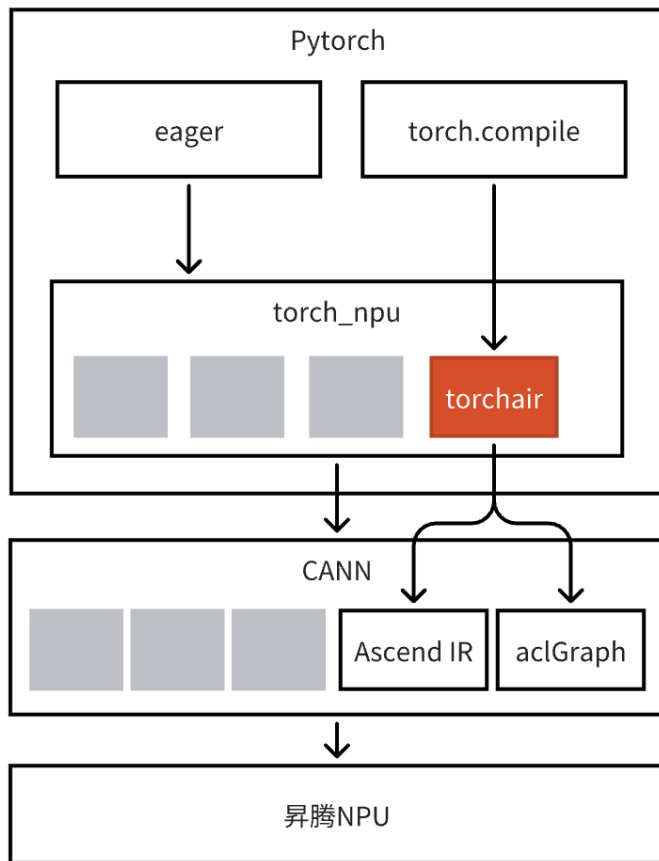


Torchair介绍 (<https://gitee.com/ascend/torchair>)

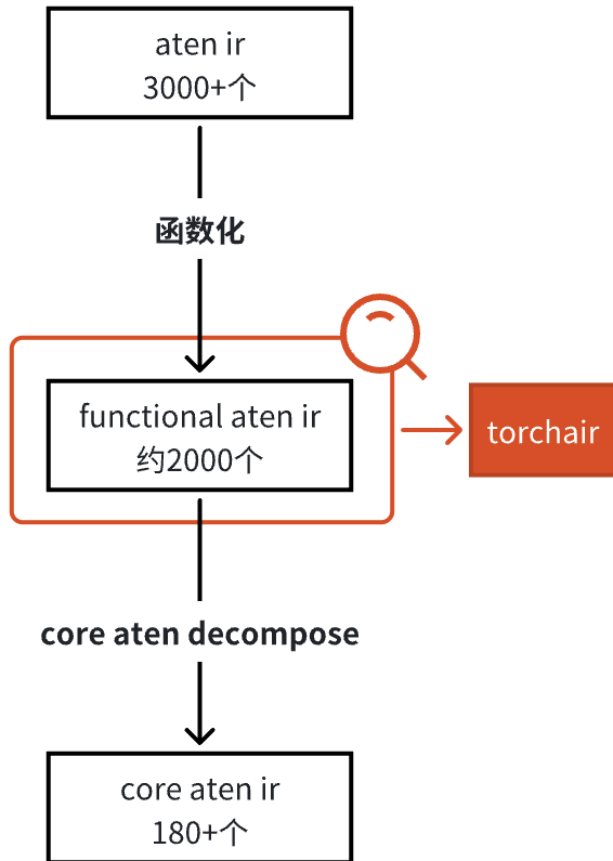
Torchair提供昇腾NPU设备亲和的
torch.compile（图模式）后端，支持
Torch用户在昇腾NPU上使用图模式进行模
型加速。

Torchair基于函数化后的aten ir（FX
Graph）工作，聚焦于调度加速、资源管理
与优化，同时进行了推理部署相关特性的
扩展与探索。

As a component:



As a compile backend:



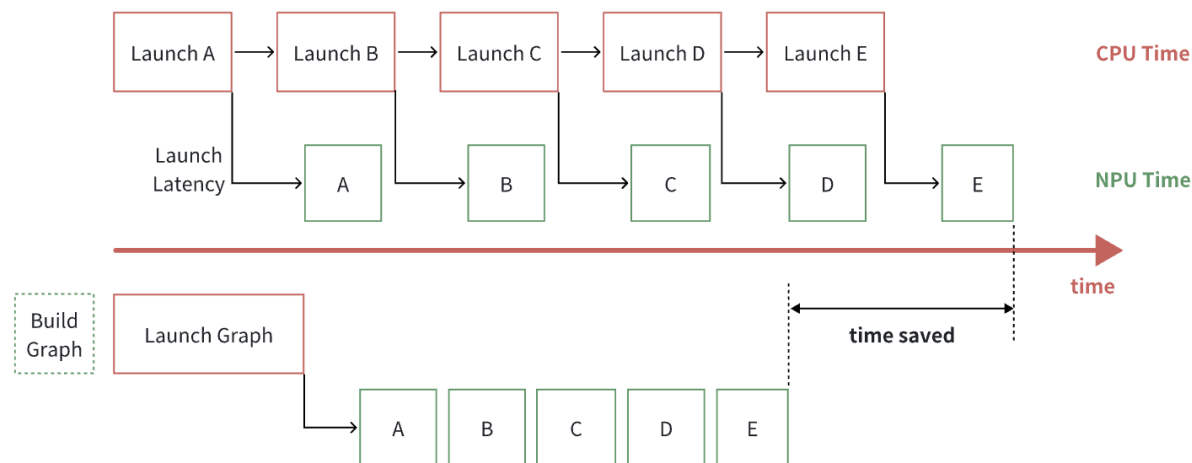
🔗 目录

- 1. 模型调度加速
- 2. 资源管理与复用
- 3. 面向推理部署的特性
- 4. 未来的工作

1. 模型调度加速

🌀 模型调度加速 - 基本原理

Kernel耗时较小时，模型执行时间主要受限于 CPU 上的 Launch 耗时。通过一次 Launch Graph 触发多个 kernel 执行（图调度），可以显著提升模型执行性能。



为什么需要torch.compile?

由于图调度要求使用相同的参数和依赖项以及内存地址运行相同的内核，因此存在许多限制。例如控制流、地址变化、参数变化等等。使用图调度需要小心地处理依赖项的变化以防止静默错误，或改造脚本以满足约束。

借助torch.compile:

- (1) 避免静默错误（如控制流）
- (2) 修改FX以满足图调度约束
- (3) 安全地处理参数的变化，更新参数或重新捕获

🌀 模型调度加速 - torchair的两种调度加速模式

Reduce-overhead模式

基于aclGraph的stream task捕获和局部task参数更新接口，提供类似cudagraph的调度加速能力，以capture+replay的方式工作，同时增强了局部参数刷新的能力。

CANN接口	功能描述
acldlIRICaptureBegin acldlIRICaptureEnd	Graph捕获（含实例化）接口
acldlIRICaptureTaskGrpBegin acldlIRICaptureTaskGrpEnd acldlIRICaptureTaskUpdateBegin acldlIRICaptureTaskUpdateEnd	基于TaskGroup的Task参数刷新接口

Max-autotune模式（Default）

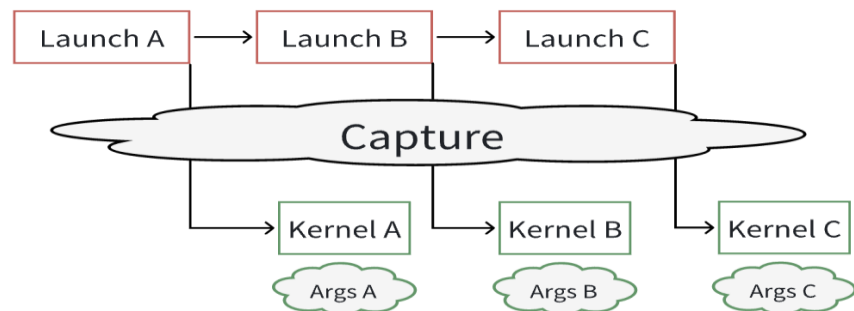
torchair将FX Graph转换为AscendIR Graph，以构图+编译+更新的方式工作。利用AscendIR Graph的模型优化与更新能力，进一步增强了对输入内存变化、参数更新、资源管理、资源复用等场景的支持。

CANN接口	功能描述
AscendIR构图接口 AddGraph CompileGraph	AscendIR Graph构图与编译接口
GetCompiledGraphSummary UpdateGraphFeatureMemoryBase RunGraphWithStreamAsync	AscendIR Graph参数刷新与执行接口

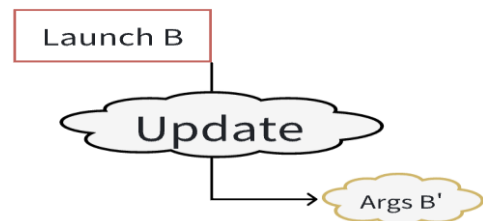
🌀 模型调度加速 - torchair的两种调度加速模式

Reduce-overhead模式

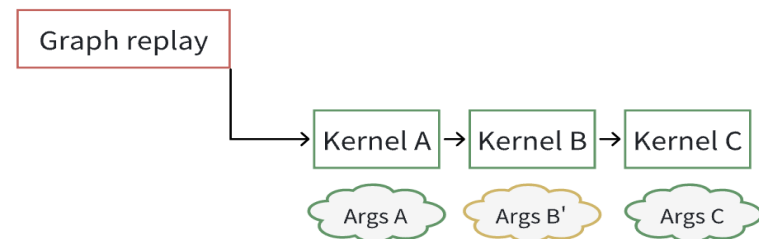
Capture阶段



运行时局部参数更新 (必要时)

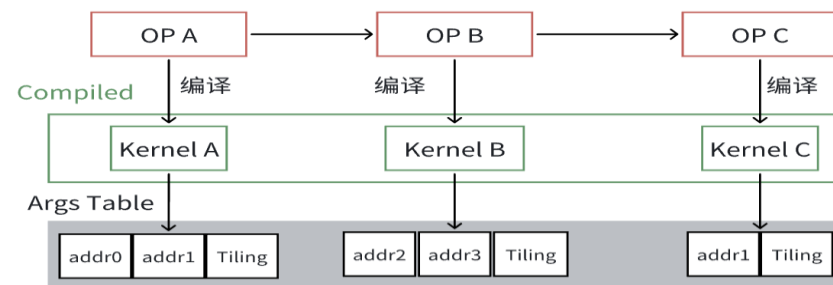


Replay执行

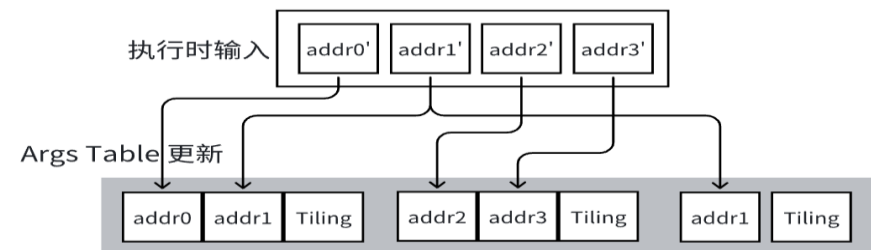


Max-autotune模式

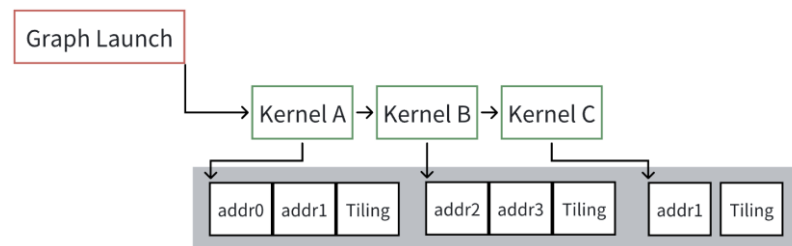
AscendIR Graph构图与编译



运行时刷新参数(必要时)



Graph Launch



🕒 模型调度加速 - torchair的两种调度加速模式

脚本使能方式:

```
import torchair
config = torchair.CompilerConfig()
config.mode = "max-autotune" # or "reduce-overhead"
@torch.compile(backend=torchair.get_npu_backend(compiler_config=config))
def forward(self):
```

...

Reduce-overhead模式内部实现

Capture iteration

```
with torch_npu.npu.graph(...) as graph:
    ops.aten.Op1(...)
    ops.aten.Op2(...)
    # Wrap task which need update in group scope
    torch.npu.graph_task_group_begin(capture_stream)
    ops.aten.Op3(Tensor, Symint)
    handle = torch.npu.graph_task_group_end(capture_stream)
    ops.aten.Op4
```

Replay iteration

```
forward(Tensors, symint...):
    torch.npu.graph_task_update_begin(update_stream, handle)
    ops.aten.Op3(Tensor, Symint)
    torch.npu.graph_task_update_end(update_stream)
    # Replay after task updated
    graph.replay()
```

Max-autotune模式内部实现

Construct AscendIR Graph

# Fx graph module	converter	# AscendIR Graph
placeholder(Tensor)	-->	ascendir.Data(Tensor)
placeholder(symint)	-->	ascendir.Data(symint)
call_function(Op1, ...)	-->	ascendir.Op1'(...)
call_function(Op2, ...)	-->	ascendir.Op2'(...)
call_function(Op3, ...)	-->	ascendir.Op3'(...)
call_function(Op4, ...)	-->	ascendir.Op4'(...)

Compile AscendIR Graph

```
session.AddGraph(id, ascendir_graph)
session.CompileGraph(id)
```

Run AscendIR Graph

```
# AscendIR Graph will update kernel args according to changed inputs
session.RunGraphWithStreamAsync(id, Tensor', Tensor'(symint))
```

🌀 模型调度加速 - Reduce-overhead模式

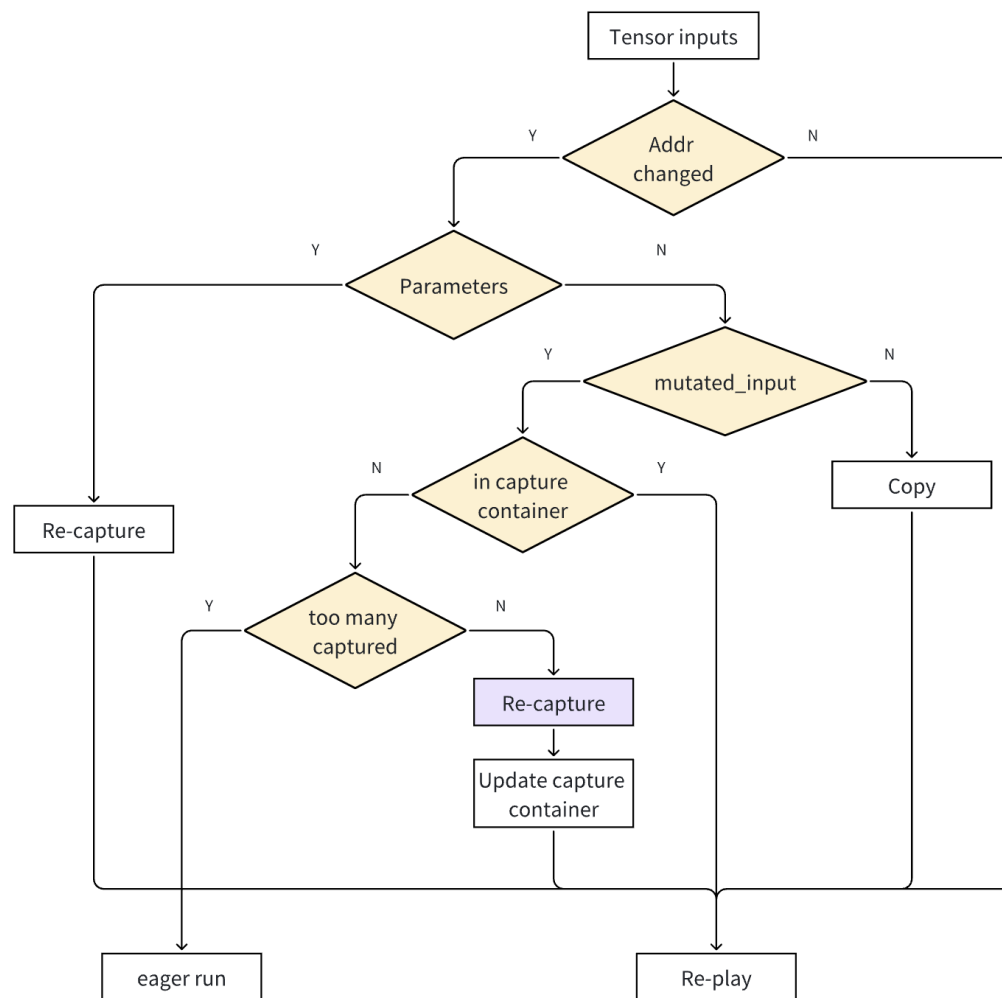
输入内存变化的处理

Capture阶段，torchair分析FX图结构以及输入信息：（1）

对于输入中的**Parameter**，直接使用其内存（而非clone新内存）用于**capture**。如果Parameter的内存地址变化（不寻常的模型），将重新capture。

（2）对于**input mutation**场景，也直接使用其内存用于**capture**，如果地址发生变化，将重新capture，并建立地址与图的存储。针对场景：对**KV Cache**的原地修改，引发的拷贝输入与输出的代价难以接受。

（3）对于其他输入的变化，则会将其数据拷贝到捕获时使用的固定内存地址上。



🌀 模型调度加速 - Reduce-overhead模式

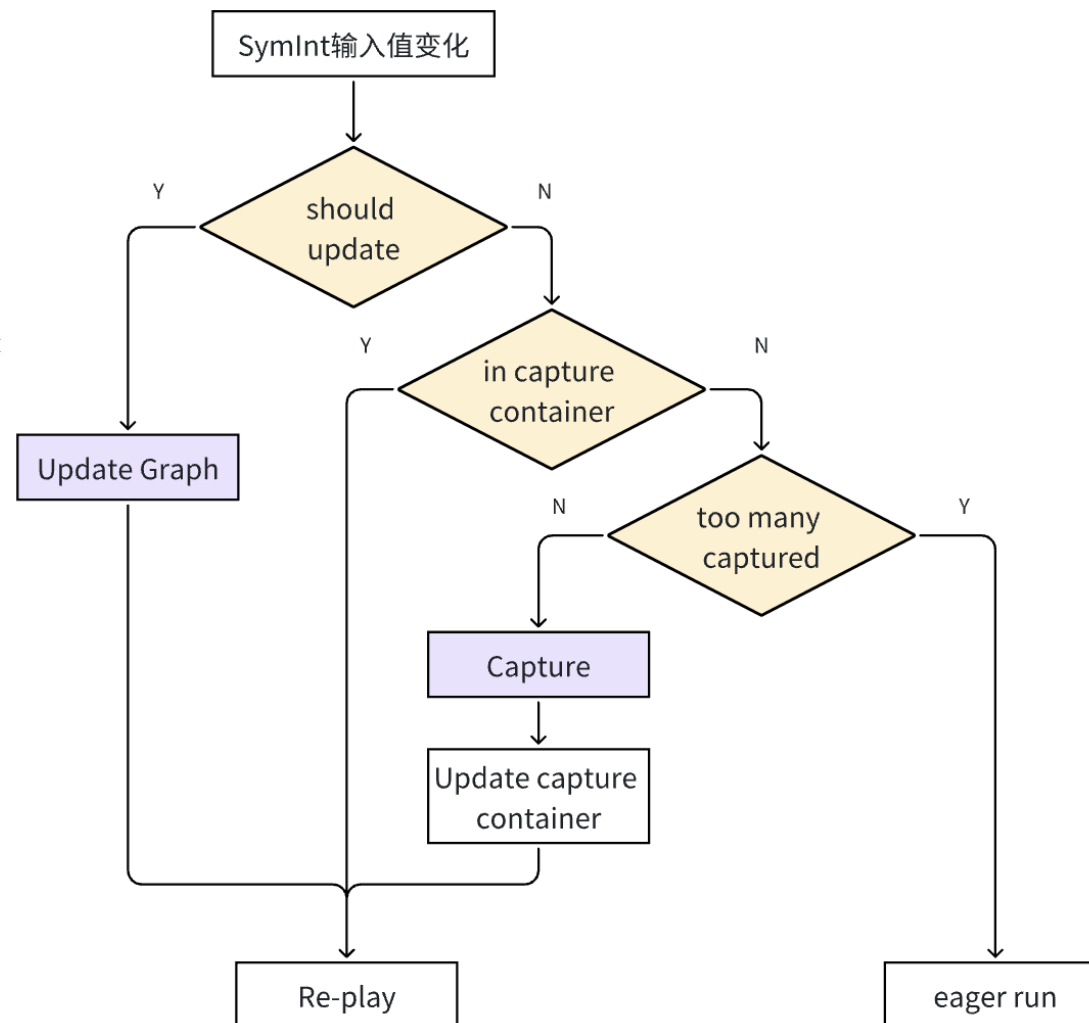
Kernel参数变化的处理

FX层的参数变化，是由于变化的Scalar输入（SymInt输入）导致，基于业务场景的策略选择：

（1）FA类的symint入参（actual_seq_length）会频繁变化，在其symint入参变化时，局部异步更新FA Kernel的参数。

（2）其他类型的参数变化（如动态shape下的Tensor维度大小变化）触发re-capture，并建立参数与捕获图的存储。

（3）当re-capture次数过多，回退至eager执行。



🌀 模型调度加速 - Max-autotune模式

Max-autotune模式下，torchair实现AtenIR到AscendIR的converter转换，将FX Graph转换为AscendIR Graph。

AscendIR Graph编译时分析输入变化影响的Kernel范围，执行时将变化的输入地址及参数更新到Graph的参数表中。

FX Graph到AscendIR Graph转换

# Fx graph module	converter	# AscendIR Graph
placeholder(arg1, Tensor)	-->	ascendir.Data(Tensor)
placeholder(s0, symint)	-->	ascendir.Data(symint)
call_function(Op1, arg1)	-->	ascendir.Op1'(...)
call_function(Op2, op1)	-->	ascendir.Op2'(...)
call_function(Op3, s0)	-->	ascendir.Op3'(...)
call_function(Op4, op2, op3)	-->	ascendir.Op4'(...)

将aten.abs转换为AscendIR abs的Converter示例

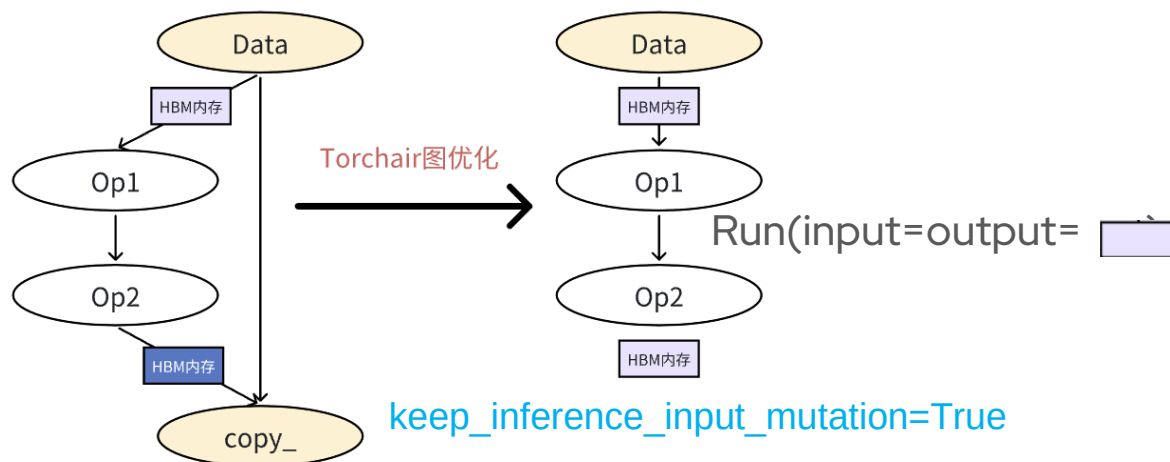
```
@register_fx_node_ge_converter(torch.ops.aten.abs.default)
def conveter_aten_abs_default(self: Tensor, meta_outputs = None):
    """NB: aten::abs(Tensor self) -> Tensor"""
    return ge.Abs(self)
```

输入地址与Symint入参变化场景处理

地址变化时刷新而非拷贝。同时，图转换时将symint转换为AscendIR的Data节点，利用AscendIR执行时的参数表更新能力完成更新。

input mutation场景处理

利用参数表刷新能力，torchair基于图分析，在图执行时传入相同的输入输出内存地址。

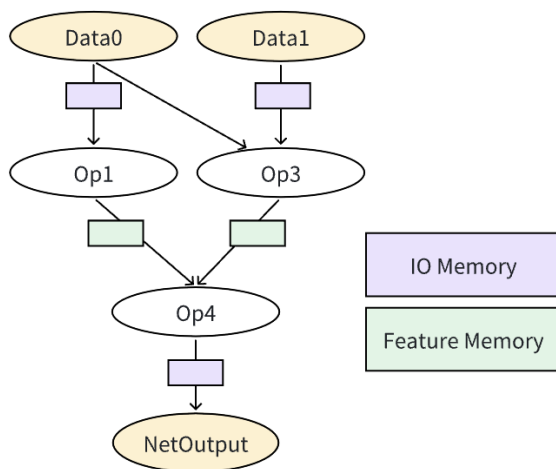


2. 资源管理与复用

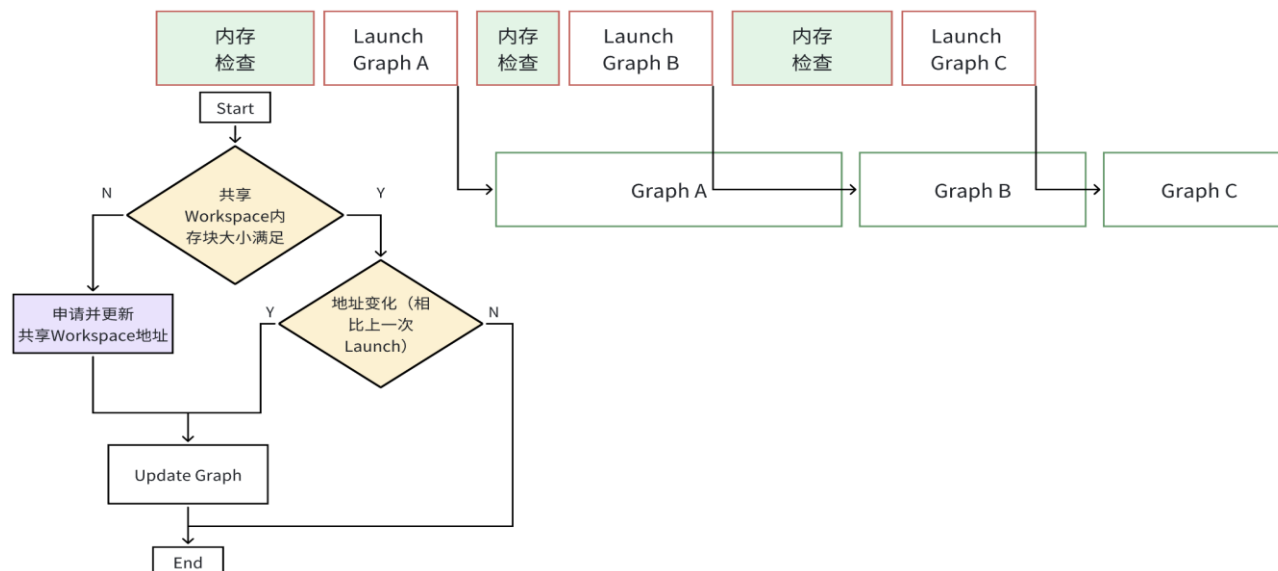
图间内存复用（Max-autotune模式）

AscendIR Graph编译结果与占用的内存资源分离。`torchair`通过查询模型所需的内存大小，外部申请内存并刷新，实现图间内存复用。

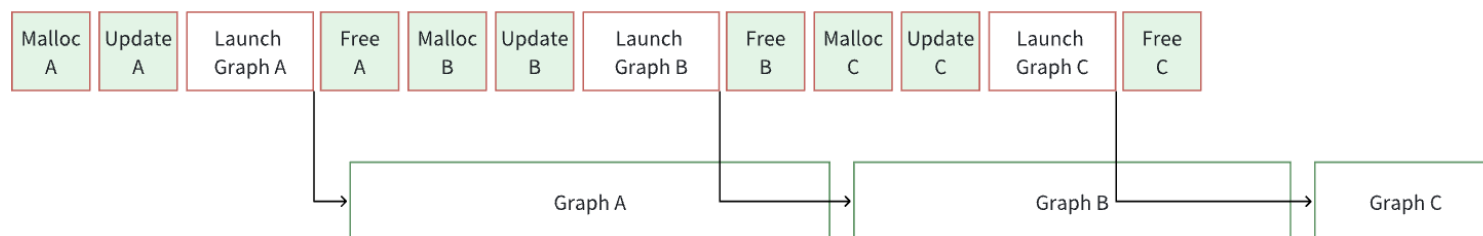
```
summary=GetCompiledGraphSummary(id)
summary->GetFeatureMemorySize(&size)
addr = Allocator->Malloc(size);
UpdateGraphFeatureMemoryBase(id, addr, size)
```



图间内存复用（默认模式，兼顾内存与刷新性能）



全局内存复用（内存高效模式）



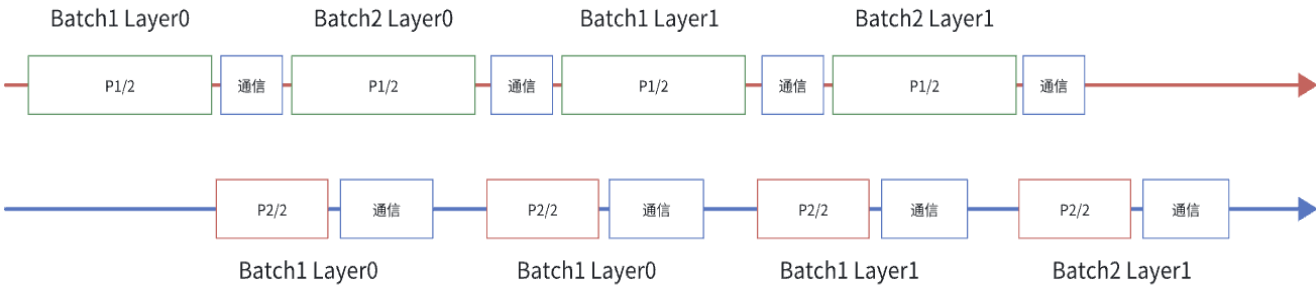
多stream并行与分核执行（Max-autotune模式）

- 1. 计算量小时，通过多stream并行+分核执行，充分利用计算资源，消除资源抢占，提升吞吐。
- 2. 在例如双MicroBatch技术中，多stream执行模型不同部分达成通信计算并发，同时分核执行，平衡并发部分的执行耗时，提升端到端性能。

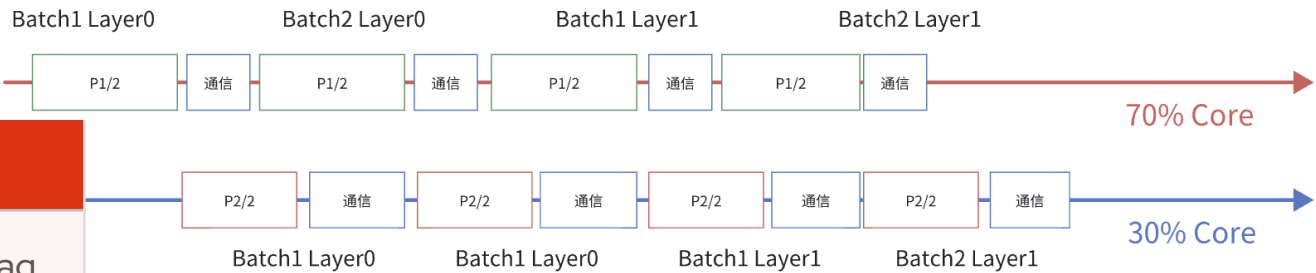
torchair提供用户在模型上标记Stream以及限制Kernel执行使用的计算核数的上下文接口：

Torchair接口	功能描述
<code>torchair.scope.npu_stream_switch</code>	上下文中的计算将在tag对应的stream上执行
<code>torchair.scope.limit_core_num</code>	上下文中的计算将最多使用配置的核数

Stream并发通信计算并行



分核执行平衡并行部分耗时



3. 面向推理部署的特性

推理部署 - torchair编译缓存

启动耗时是torch.compile的关键问题之一。torchair实现基于字节码的编译缓存，在推理弹性扩容等场景，将启动耗时降低至ms级。

缓存脚本到编译结果的全流程



模型脚本

```
def forward(self,
    x: InputMeta,
    kv: List[torch.Tensor]):
    raw_python_func(x)
    layer(...)
    raw_python_func(...)
```

编译结果PYTHON字节码

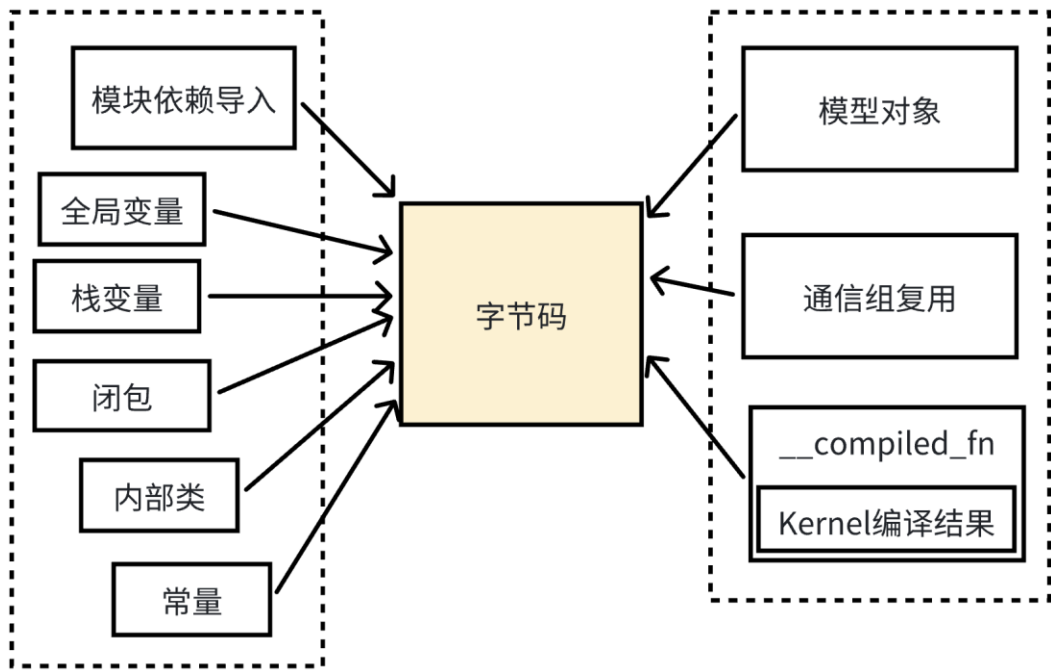
```
LOAD_GLOBAL 0
LOAD_METHOD
LOAD_FAST
CALL_FUNCTION compiled_fn0
CALL_XX
RETURN VALUE
```

Torchair接口	功能描述
torchair.inference.cache_compile	进行compile编译，同时管理编译结果字节码缓存。

缓存编译后的字节码至本地，模型forward时触发缓存加载，恢复字节码执行所需的Frame信息。同时注入外部通信组、模型对象等实例资源，恢复模型执行。

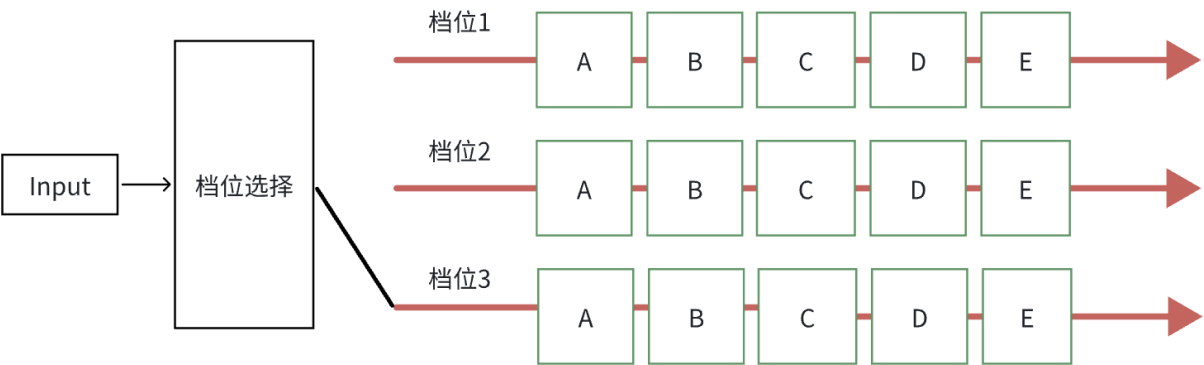
Frame恢复

运行时资源注入



推理部署 - 分档编译（Max-autotune模式）

推理时通常会批量执行请求以提高计算资源利用率，对于batch大小在特定的一组值中变化的场景。torchair提供在Tensor上标记动态batch维度及可选档位的能力，利用AscendIR Graph的分档编译能力，实现动态多档位模型的下沉执行以及在多档位间共享内存资源能力。



使能分档编译示例

```
model = torch.compile(Model(), backend=npu_backend, dynamic=True)
x = torch.ones([32, 16])
torchair.inference.set_dim_gears(x, {0: [32, 64]})
model(x)
# Will run directly as compiled graph support 64 dim[0]
y = torch.ones([64, 16])
```

Torchair接口	功能描述
torchair.inference.set_dim_gears	设置Tensor在对应维度上的取值范围。

4. 未来的工作

🕒 未来的工作

1. Reduce-overhead模式多图内存复用
2. Reduce-overhead模式Stream并行与分核执行
3. Reduce-overhead模式Kernel静态化编译
4. Reduce-overhead模式下的自动分档能力
5. 推理场景内部格式权重加速

PyTorch Ecosystem on Ascend

The PyTorch logo is a large, light purple graphic on the right side of the slide. It consists of a stylized flame or drop shape with a circular element at the top right, all enclosed within a larger circular outline.

Yuanhao Ji @shink
jiyuanhao@apache.org



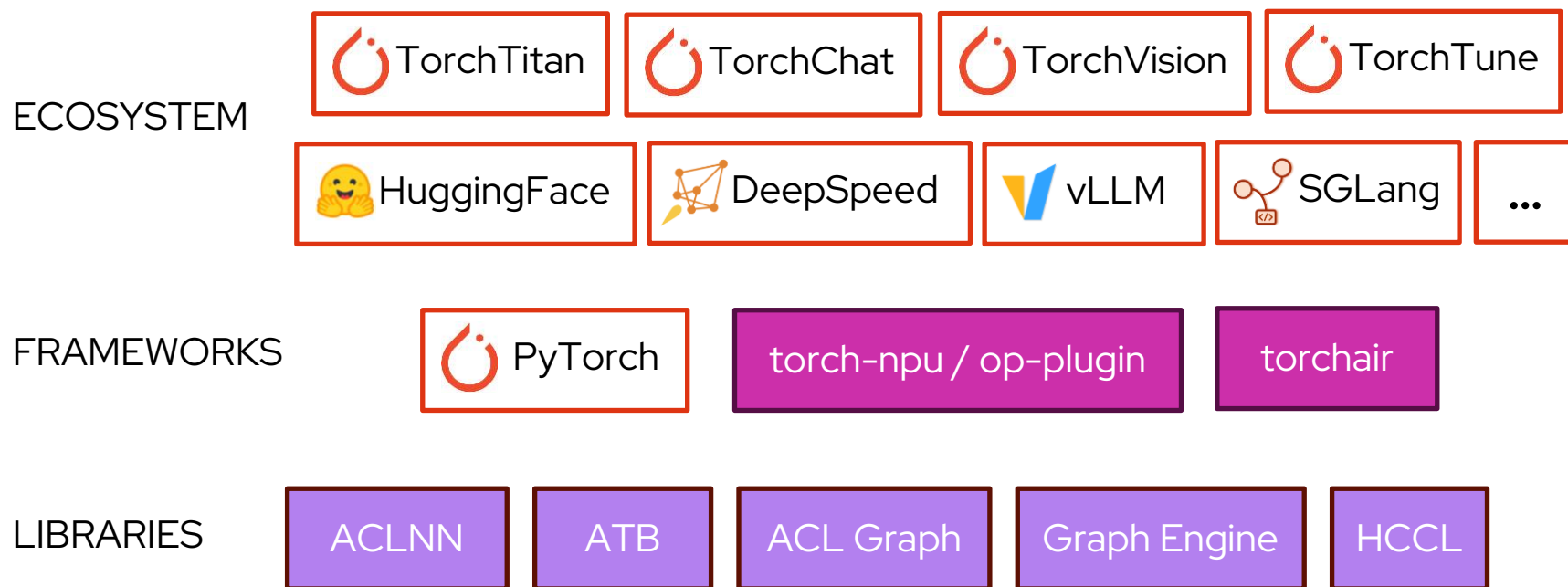
Content

- PyTorch Ecosystem
- Training
- Fine-tuning
- Inference
- Future

PyTorch Ecosystem

A large, faint, light purple PyTorch logo watermark is visible in the background on the right side of the slide. The logo consists of a stylized 'P' formed by two concentric arcs and a small circle at the top right.

PyTorch Ecosystem on Ascend



 PyTorch x  Ascend

🔥 PyTorch Ecosystem on Ascend



Training



TorchTitan



A PyTorch native platform
for training generative AI models

Github URL:  3.8k

<https://github.com/pytorch/torchtitan>



**COMPOSABLE
N-D
PARALLELISM
TRAINING**

FSDP/TP/SP/PP



**DISTRIBUTED
CHECKPOINT**

Fine-tuning with
torchtune



**EFFICIENT
TRAINING
TECHNIQUES**

Activation
recompute
FP8 Training
torch.compile

TorchTitan on Ascend

并行能力

- ☒ FSDP
- ☒ TP
- ☒ CP
- ☒ PP
- ☒ SP
- ☒ DDP

分布式检查点

- ☒ Selective
- ☒ Full

图模式

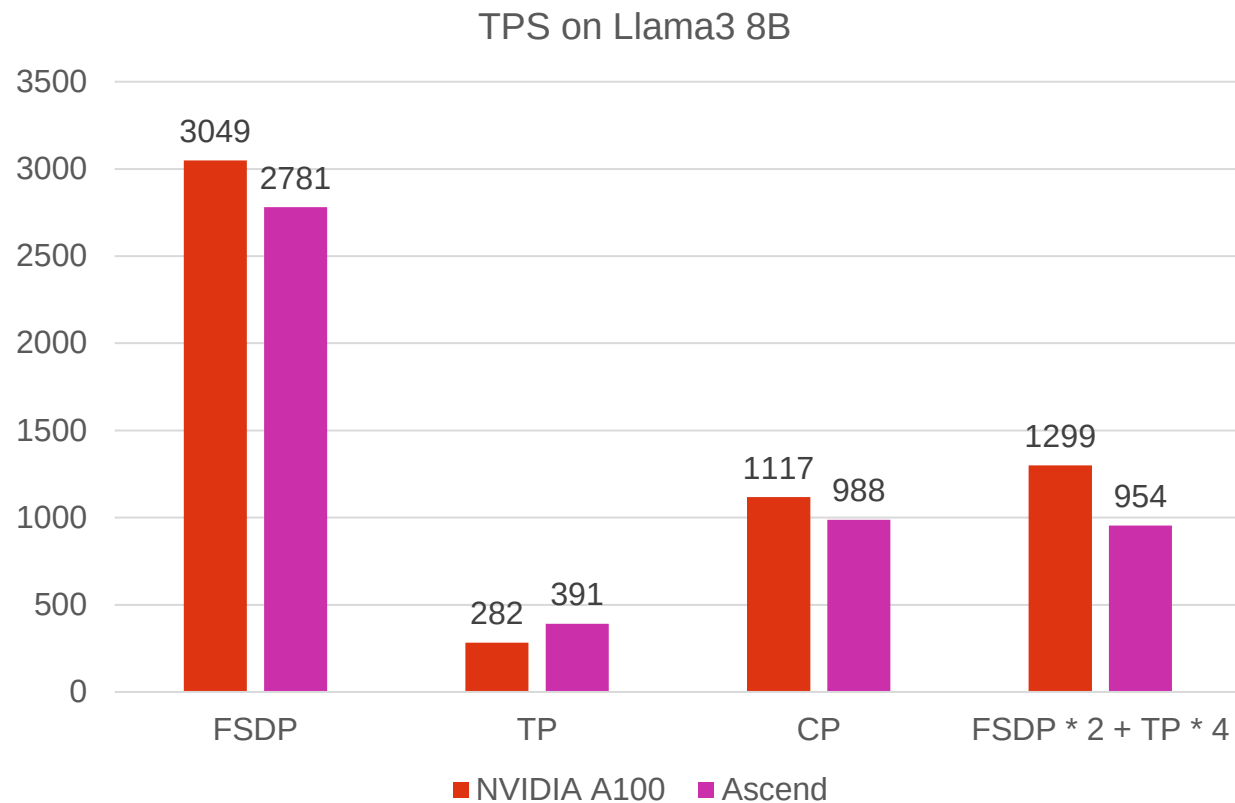
- ☐ torch.compile

数据类型

- ☐ Float8
- ☒ BFloat16
- ☒ Float16
- ☒ Float32

- Support 3rd-party backend – torchtitan
- Support 3rd-party distributed backend – torchtitan
- ...

TorchTitan on Ascend



使用说明:

https://ascend.github.io/docs/sources/torchtitan/quick_start.html

Fine-tuning



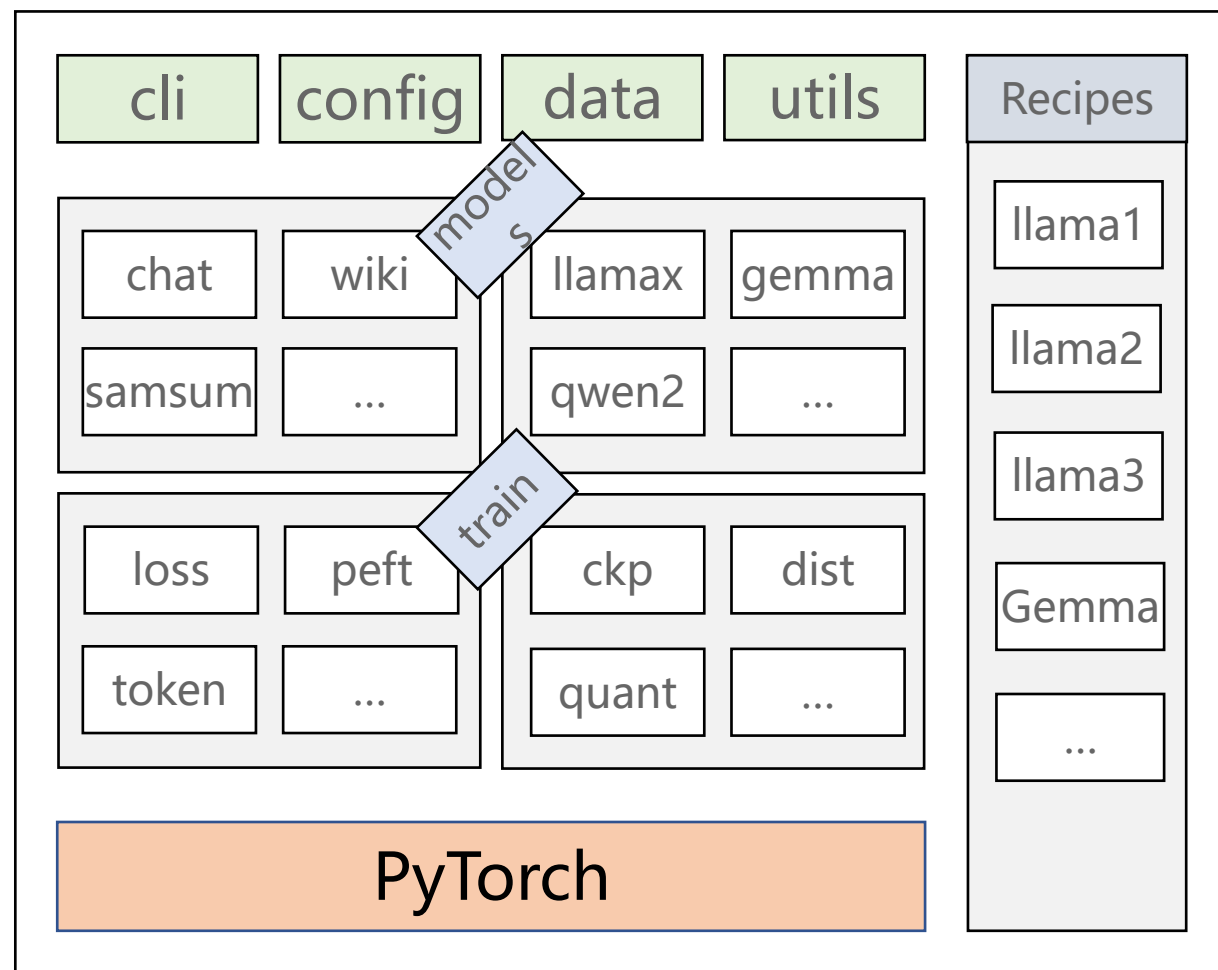
TorchTune



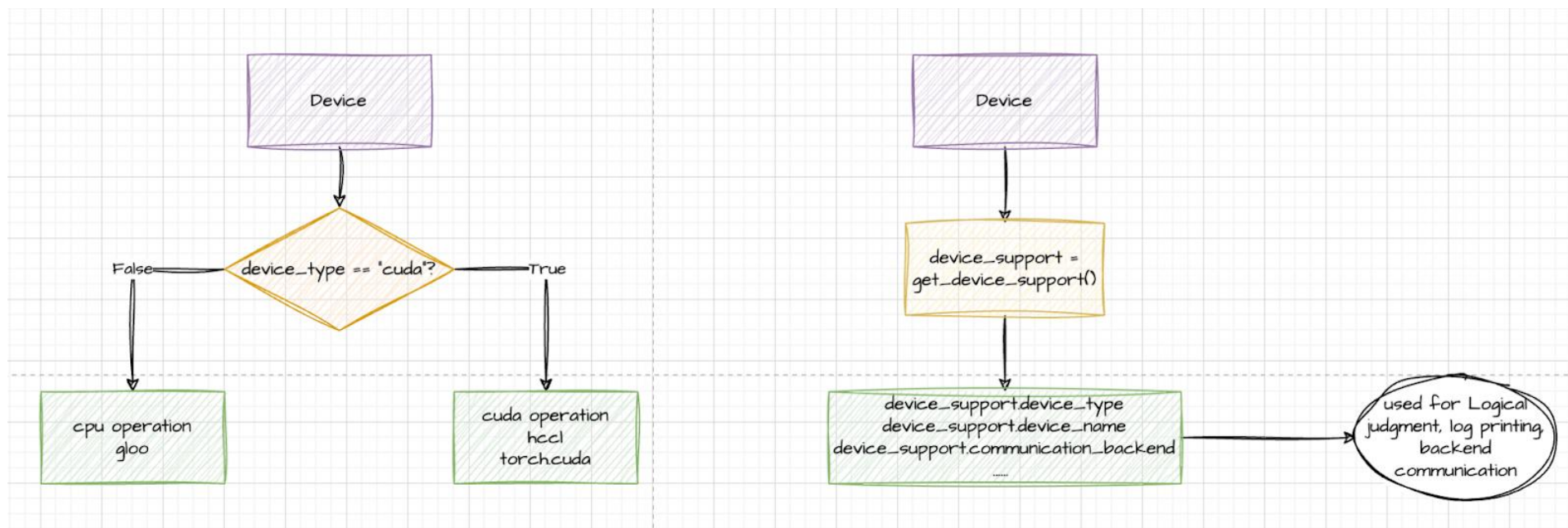
torchtune is a PyTorch library for easily authoring, post-training, and experimenting with LLMs

Github URL:  5.2k

<https://github.com/pytorch/torchtune>



TorchTune on Ascend



- Add Ascend NPU as a backend – torchtune
- Add Ascend NPU as a backend for single device recipes – torchtune
- Add Ascend NPU as a backend for distributed recipes – torchtune
- ...

Inference





vLLM is a fast and easy-to-use library
for LLM inference and serving.

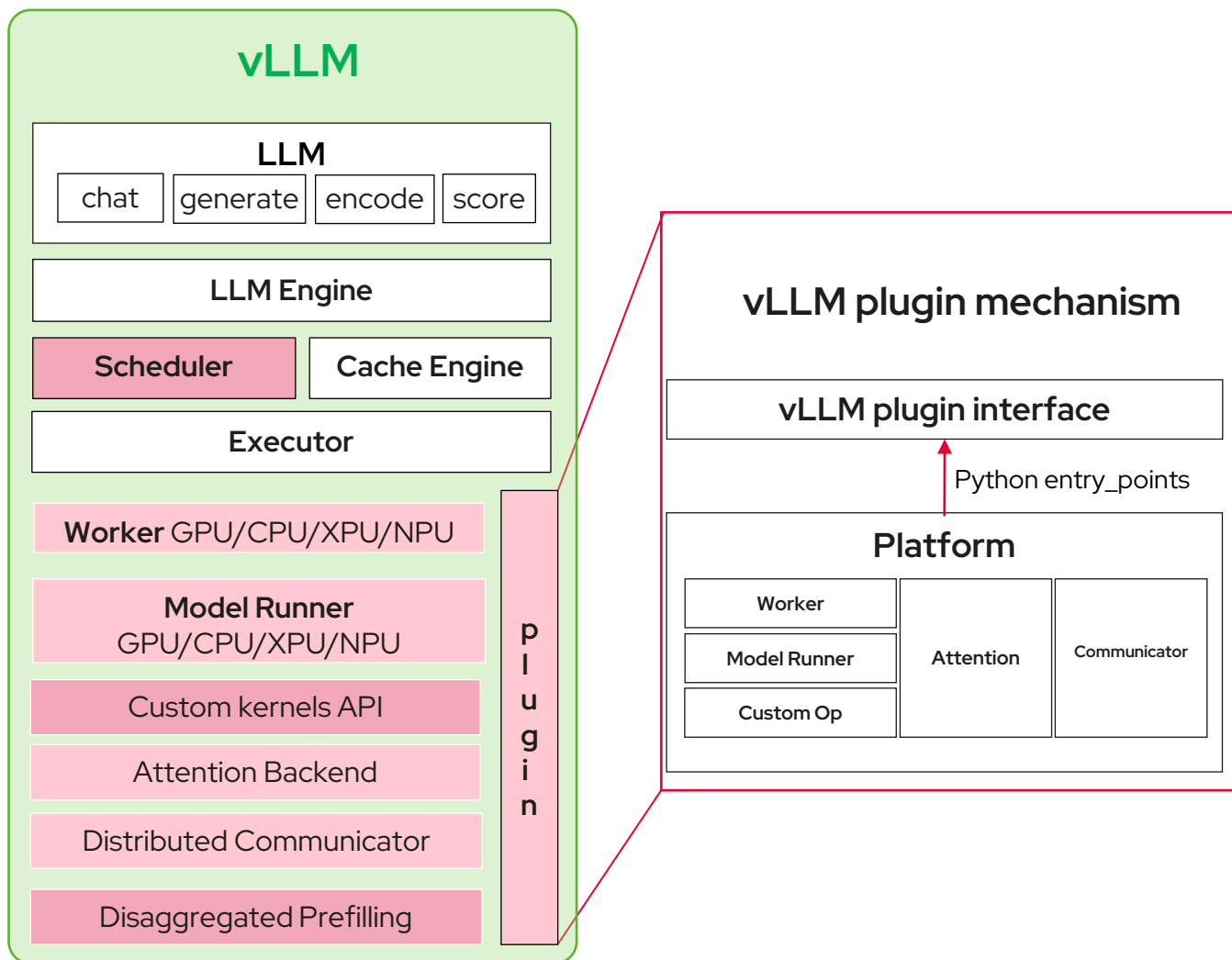
Github URL:



<https://github.com/vllm-project/vllm>

- 100+ supported models
- 23+ supported quantization methods
- 8+ supported hardware
- 100+ supported custom ops
- 12+ supported attention backends

vLLM on Ascend



Feature	vLLM V0 Engine	vLLM V1 Engine
Chunked Prefill	🚧 WIP	● Functional
Automatic Prefix Caching	🚧 WIP	● Functional
LoRA	● Functional	🚧 WIP
Pooling	● Functional	● Planned
Tensor Parallel	● Functional	● Functional
Pipeline Parallel	● Functional	● Functional
Expert Parallel	● NO plan	● Functional
Data Parallel	● NO plan	● Functional
Quantization	● Functional	● Functional
Graph Mode	● NO plan	● Experimental

vLLM on Ascend

```
# 安装 torch
$ pip install torch

$ pip install torch-npu # Only for Ascend

# 安装 vllm
$ pip install vllm

$ pip install vllm-ascend # Only for Ascend
```

```
from vllm import LLM, SamplingParams

# 用户输入
prompts = [
    "Hello, my name is",
    "The president of the United States is",
    "The capital of France is",
    "The future of AI is",
]

# 初始化vLLM, 载入模型
llm = LLM(model="Qwen/Qwen2.5-0.5B-Instruct")

# 执行推理
outputs = llm.generate(prompts, sampling_params)

# 打印输出
for output in outputs:
    prompt = output.prompt
    generated_text = output.outputs[0].text
    print(
        f"Prompt: {prompt!r},\n"
        f"Generated text: {generated_text!r}"
    )
```

Future





Future

- More projects
- `torch.compile`

Thanks

